Courses Programmierpraktikum QR-Code  Help Lucia Ludwig (s451439) (Logou

PABS 3.12.1 - University of Würzburg - Impressum - Datenschutz

QR-Code
Ein QR-Code (Quick Response – Code) ist ein zweidimensionaler optischer Barcode, der z.B. auch dazu verwendet wird, Weblinks zu kodieren. Der folgende QR-Code enthält beispielsweise den Link zur WueCampus Webseite.

Er wurde ursprünglich für die Automobilindustrie Japans entwickelt und besteht aus schwarzen und weißen Quadraten, Module genannt, die in einer quadratischen Matrix angeordnet sind. QR-Codes unterstützen vier verschiedene Zeichencodierungen: numerisch, alphanumerisch, Byte-Kodierung sowie die Kanji-Zeichen, die im japanischen Schreibsystem verwendet werden. Aufgrund der eingebauten Fehlerkodierung, kann ein QR-Code auch dann noch korrekt gelesen werden, wenn ein Teil davon fehlt.

Hinweis: Es sind alle Klassen und Funktionsdeklarationen bereits gegeben, damit die Tests kompilieren. Entfernen Sie einfach aus den Methoden den Exception-throw und fügen Sie Ihre Implementation ein.

Einführung: Bitoperationen
Da in dieser Aufgabe einiges mit Bitoperationen gemacht wird, wird im Folgenden kurz erklärt, wie man einzelne Bits aus einer Zahl setzt bzw. ausliest. Die vorgestellten Methoden scheinen auf den ersten Blick zwar sehr kompliziert, sind aber die mit Abstand effizientesten Methoden, da sie meist in 3-4 Assemblerinstruktionen resultieren.
Machen Sie sich damit vertraut, sie können auch in anderen Aufgaben noch zu gebrauchen sein (und vor allem in späteren Projekten!).

Auslesen

Zum Auslesen des i -ten Bits (Nullindiziert von rechts) aus der Zahl n geht man folgendermaßen vor:
Zuerst isoliert man das gesuchte Bit.

IsolatedBit = n & (1 << i)

Diese Verbindung mit dem binären AND nennt man "maskieren", wobei die Zahl (1 << i) Maske heißt (hat nichts mit der Maske des QR-Codes zu tun). Das liegt daran, dass das Ergebnis des ANDs genau noch die Bits enthält, an deren Position die Maske eine 1 hatte. Wenn man also z.B. die letzten vier Bits einer Zahl maskieren möchte, nutzt man die Maske

0b1111.

Um nun zu überprüfen, ob das Bit gesetzt ist, muss man IsolatedBit nur noch auf != 0 überprüfen.

Result = IsolatedBit != 0

Oder in einer Zeile:

Result = (n & (1 << i)) != 0

Beispiel: Gesucht ist das Bit am Index 1 der Zahl 10 = 0b1010.
Also gilt für IsolatedBit: IsolatedBit = 10 & (1 << 1) = 0b1010 & 0b0010 = 0b0010. Der Wert des Bits ist also (0b0010!= 0) = 1.

Man kann diese Operationen natürlich auch in Schleifen nutzen, so kann man z.B. mit dem folgenden Code alle Bits aus dem byte[] ausgeben:

byte[] bytes = new byte[] { 1, 2, 3, 4, 5, ... };
int bitOffset = 7;
int offset = 0;
while(offset < bytes.length) {
boolean bit = ((bytes[offset] >> bitOffset) & 1) != 0;
System.out.print(bit ? '1' : '0');
if(bitOffset == 0) {
bitOffset = 7;
offset++;
} else {
bitOffset--;
}
}

Schreiben

Zum Setzen des i -ten Bits (Nullindiziert von rechts) in der Zahl n wird ein ähnliches Vorgehen gewählt. Zuerst wird das zu setzende Bit an der richtigen Position erzeugt:

BitToSet = (1 << i)

Dann wird diese Zahl mit n mit einem binären OR verbunden:

Result = n | BitToSet

Oder in einer Zeile:

n = n | (1 << i) bzw. n |= (1 << i)

Man kann natürlich auch mehrere Bits mit einem OR setzen, dazu müssen in BitToSet eben alle Bits, die gesetzt werden sollen, gesetzt sein.

Beispiel: Setze das Bit am Index 2 in der Zahl 10 = 0b1010. Also gilt BitToSet = (1 << 2) = 0b100 und Result = n | BitToSet = 0b1010 | 0b0100 = 0b1110 = 14

Informationen zu QR Codes

Allgemein

QR-Codes existieren in den Versionen 1-40, die unter anderem die Größe des Codes anzeigen. Ein QR-Code ist n x n Module groß, mit n = 17 + 4 * Versionsnummer (Version 1 ist dementsprechend 21x21 Module groß, Version 2 25x25 Module, usw.). Die Module werden reihen- und spaltenweise indiziert. Das Modul (0,0) ist in der linken oberen Ecke. Die Koordinaten (i,j) entsprechen dem Modul in der i -ten Reihe und j -ten Spalte. Ein schwarzes Modul entspricht einem 1 Bit, ein weißes Modul dementsprechend einem 0 Bit. Da Bitreihen in Zahlen umgewandelt werden, ist es notwendig, sich mit dem Begriff Bitwertigkeit vertraut zu machen. Im Folgenden steht MSB für Most Significant Bit und LSB für Least Significant Bit.

Um den Code einfacher zu scannen zu machen, muss gewährleistet sein, dass ein QR-Code in etwa die gleiche Anzahl an schwarzen und weißen Modulen besitzt. Dazu werden Masken verwendet, die die Farbe bestimmter Modulen im Code invertiert. Sie wird ausschließlich auf Module angewandt, die die eigentlichen Daten enthalten.

Außerdem haben QR-Codes eine Fehlerkorrekturmethode eingebaut. Die Daten werden als Reed-Solomon-Code codiert und können beim Dekodieren auf Fehler überprüft werden und korrigiert werden (dies ist schon für sie implementiert). Hierbei gibt es 4 Fehlerkorrekturlevel, mit denen man die Korrekturfähigkeit bestimmen kann, wobei mit steigender Fehlertoleranz mehr Korrekturbits gebraucht werden, also weniger Daten im Code gespeichert werden können.

Layout
Da ein QR-Code ein optischer Code ist, der mit einer Kamera aufgenommen und dann mithilfe von bildverarbeitenden Methoden in die entsprechende Matrix umgewandelt wird, enthält der Code Elemente, die zur Ausrichtung des Codes notwendig sind. Die Struktur eines QR-Codes nach ISO Standard ist hier zu sehen:

Zur einem korrekten QR-Code gehören also (siehe Modulplatzierung für detaillierte Beschreibung):

Orientation Patterns (rot) in allen Ecken außer unten rechts
Separators (dunkelgrün) um die Orientation Patterns
Timing Patterns (orange) zwischen den Orientation Patterns, damit der Bildverarbeitungsprozess die Größe eines einzelnen Quadrates erkennt
Dark Module
Alignment Patterns (rosa), um auch verzerrte QR-Codes dekodieren zu können
2x redundante Format-Information (blau), enthält die verwendete Maske und das Fehlerkorrekturlevel
2x redundante Version-Information (grau), enthält die Version dieses QR-Codes (zum Überprüfen)

Modulplatzierung
Orientation Pattern

Wie schon erwähnt sind die Orientation Patterns immer in allen Ecken außer unten rechts. Sie sehen folgendermaßen aus:

Außerdem muss der Bereich um die Patterns immer weiß bleiben.

Timing Patterns

Die Timing Patterns verbinden die inneren Ecken der Orientation Patterns, die vertikale Timing Pattern ist also in Reihe 6 und die horizontale in Spalte 6. Sie sind abwechselnd schwarz und weiß und starten immer außerhalb des Separators mit einem schwarzen Modul.

Dark Module

Das Dark Module ist an der Position (n-8,8) und ist schwarz.

Alignment Patterns

Die Alignment Patterns befinden sich an für jede Version vordefinierten Positionen. Für jede Version sind bestimmte Werte vorgegeben, z.B. für die Version 8 die Zahlen 6, 24, 42. Die Kombinationen aus diesen Zahlen bestimmen mögliche Positionen der Alignment Patterns. Positionen, bei denen das Alignment Pattern über einem Orientation Pattern liegt, werden verworfen, es werden also (6,6), (6,42), (42,6) verworfen, die übrigen Patterns erkennt man in der unteren Grafik.

Alignment Patterns können auf Timing Patterns liegen, was aber nichts macht, da diese genau so positioniert sind, dass sich die Muster nicht behindern.

Format-Information

Die Format-Information liegt in allen Versionen redundant um die Orientation Patterns nach der unteren Grafik verteilt, d.h. die Bits 0-14 um den linken oberen Orientation Pattern bilden die erste Kopie der Format-Information und die restlichen die zweite. Das Bit 14 ist das MSB und das Bit 0 das LSB der 15 Bit Format-Information.

Diese Werte sind bereits vorhanden, siehe Erklärung zur Klasse FormatInformation.

Version-Information

Die Version-Information existiert nur für Versionen > 6 und liegt redundant in den in der Grafik blau markierten Bereichen.

Die obere Version-Information wird für jede Reihe (von oben nach unten) von links nach rechts ausgewertet. Oben links ist das LSB und unten rechts das MSB. Die untere Version-Information ist ähnlich gespeichert, nur dass man für jede Spalte (von links nach rechts) von oben nach unten geht. Das LSB ist wieder oben links und das MSB unten rechts. Hier ein Beispiel für einen Version 7 QR-Code, der die Version-Information 0b000111110010010100 hat.

Diese Werte sind bereits vorhanden, siehe Erklärung zur Klasse VersionInformation.

Daten

Die Datenbytes sind in allen verbleibenden Modulen platziert, jeweils vom MSB zum LSB. Sie sind nach dem Schlangenmuster unten in Zweierreihen sortiert. Sämtliche vorher genannten Patterns werden von diesem Prozess übersprungen. Wie man außerdem sieht, wird die Spalte mit dem Index 6 komplett übersprungen, da sie sowieso von Timing bzw. Orientation Pattern vollständig belegt ist. Damit geht auch die ungerade Seitenlänge des QR-Codes perfekt auf.

Innerhalb dieser Zweierreihen sind die Bits von rechts nach links platziert:

Einzelne Bytes müssen nicht in perfekten 8er Blöcken zusammen liegen, weil z.B. ein Alignment-Pattern ein paar Module blockiert. Das Zickzackmuster wird an diesen Stellen einfach fortgesetzt und immer, wenn wieder ein Feld frei ist, liegt dort das nächste Bit.

Formatierung der Daten
Die Daten werden folgendermaßen für einen QR-Code codiert:

Codieren nach einem gewissen Data Encoding
Aufteilen in Blöcke und berechnen der Korrekturbits
Vermischen dieser Blöcke

Das Beispiel am Ende jedes der folgenden Absätze bezieht sich immer auf die Kodierung der Nachricht "Hello there." - "General QRCodi. You are a square one." mit dem Korrekturlevel Quartile.

Codierung

Es gibt einige Kodierungen, z.B. Numerisch, Kanji, Alphanumerisch und Byte. Wir werden uns im Folgenden auf die Bytekodierung beschränken, weil sie am einfachsten zu implementieren ist und man mit ihr am besten Nachrichten codieren kann.
Alle anderen Formatierungsschritte sind komplett unabhängig von der gewählten Kodierung, man könnte das Projekt an dieser Stelle also leicht erweitern.

Für die Byte-Codierung wird die Nachricht mit ISO-8859-1 kodiert. Diese Bytes werden im Folgenden als Daten bezeichnet.
Die Ausgabe dieses Schritts sieht dann folgendermaßen aus:

Indicator Character count indicator Data Terminator

0b0100 8 bzw. 16 Bit uint (Little Endian) n Bytes 0000 (4 bits)

Bis Version 10 werden 8 Bits für den Character count indicator (Cci) genutzt und ab (einschließlich) Version 10 16 Bits.

Wenn die Bitanzahl kein Vielfaches von 8 ist, sollen bis zum nächsten Vielfachen von 8 mit 0 aufgefüllt werden. Wenn jetzt immer noch nicht alle Datenbytes für diese Version und das gewünschte Fehlerkorrekturlevel aufgebraucht sind, wird mit den Padding Bytes 236 und 17 abwechselnd aufgefüllt.

Beispiel:

"Hello there." - "General QRCodi. You are a square one." in ISO-8859-1 kodiert: 22 48 65 6C 6C 6F 20 74 68 65 72 65 2E 22 20 2D 20 22 47 65 6E 65 72 61 6C 20 51 52 43 6F 64 69 2E 20 59 6F 75 20 61 72 65 20 61 20 73 71 75 61 72 65 20 6F 6E 65 2E 22 (56 Bytes)

Länge der Daten (Bits): 4 (Indicator) + 8 (Cci) + 8 * 56 + 4 (Terminator) = 464 (ist bereits Vielfaches von 8) => 58 Bytes
Kleinste Version für 58 Bytes mit Quartile: Version 5 mit 62 Bytes
Version < 10 => keine Korrektur der Länge des Cci nötig
62 - 58 = 4 Padding Bytes

Indicator Cci Data Terminator Padding

0100 00111000 00100010 01001000 01100101 01101100 01101100 01101111 00100000 01110100 01101000 01100101 01110010
01100101 00101110 00100010 00100000 00101101 00100000 00100010 01000111 01100101 01101110 01100101
01110010 01100001 01101100 00100000 01010001 01010010 01000011 01101111 01100100 01101001 00101110
00100000 01011001 01101111 01110101 00100000 01100001 01110010 01100101 00100000 01100001 00100000
01110011 01110001 01110101 01100001 01110010 01100101 00100000 01101111 01101110 01100101 00101110
00100010

0000 11101100
00010001
11101100
00010001

Ergebnis:

01000011 10000010 00100100 10000110 01010110 11000110 11000110 11110010 00000111 01000110 10000110 01010111 00100110 01010010 11100010 00100010 00000010 110 10010 00000010 00100100 01110110 01010110 11100110 01010111 00100110 00010110 11000010 00000101 00010101 00100100 00110110 11110110 01000110 10010010 111000 10 00000101 10010110 11110111 01010010 00000110 00010111 00100110 01010010 00000110 00010010 00000111 00110111 00010111 01010110 00010111 00100110 01010010

00000110 11110110 11100110 01010010 11100010 00100000 11101100 00010001 11101100 00010001 (Bin)

43 82 24 86 56 C6 C6 F2 07 46 86 57 26 52 E2 22 02 D2 02 24 76 56 E6 57 26 16 C2 05 15 24 36 F6 46 92 E2 05 96 F7 52 06 17 26 52 06 12 07 37 17 56 17 26 52

06 F6 E6 52 E2 20 EC 11 EC 11 (Hex)

Aufteilen in Blöcke

Als nächstes wird die Ausgabe des letzten Schritts in Blöcke aufgeteilt. Ab hier ist alles für alle Encodings gleich. Dazu gibt es für jede Kombination aus einer Version mit einem Fehlerkorrekturlevel unterschiedliche Vorgaben, die Ihnen die Version Klasse liefert.
In einem Block sind immer eine bestimmte Byteanzahl Daten und die dazugehörigen Korrekturbytes. Die Anzahl Korrekturbytes sind für alle Korrekturblöcke einer Version gleich. Außerdem gibt es sog. Gruppen, die eine bestimmte Anzahl Blöcke mit einer bestimmten Anzahl Datenworte definieren. Von diesen Gruppen gibt es jedoch max. 2 pro Version und die Blöcke der zweiten Gruppe enthalten immer (wenn sie existiert) 1 Datenwort mehr als die der ersten Gruppe.

Beispielsweise ist für Version 5-Q (also Version 5 mit Fehlerkorrekturlevel Quartile) gegeben:

EC Bytes Per
Block

Number of Blocks in
Group 1

Number of Data Bytes in Each of Group 1's
Blocks

Number of Blocks in
Group 2

Number of Data Bytes in Each of Group 2's
Blocks

18 2 15 2 16

Die Blöcke werden in der Reihenfolge ihrer Gruppe mit Datenbytes gefüllt, also die ersten 15 Datenbytes gehen in Block 1, die nächsten 15 in Block 2, die nächsten 16 in Block 3 und die letzten 16 in Block 4. Außerdem werden für diese Blöcke jeweils 18 Korrekturbytes mit Reed-Solomon berechnet, die die Korrekturbytes für den jeweiligen Block bilden.

Beispiel

Block 1:

Datenbytes 43 82 24 86 56 C6 C6 F2 07 46 86 57 26 52 E2
Korrekturbytes 60 B9 88 5F AD 14 E9 C9 61 44 8E 96 57 C6 74 A3 E3 CE

Block 2:

Datenbytes 22 02 D2 02 24 76 56 E6 57 26 16 C2 05 15 24
Korrekturbytes 31 21 4F 43 B3 46 AD 68 7D 09 AC 7A 20 AE F2 C7 C8 7E

Block 3:

Datenbytes 36 F6 46 92 E2 05 96 F7 52 06 17 26 52 06 12 07
Korrekturbytes A3 FC 44 AD 2C 39 B1 7E 08 2F AA 77 6B 47 2F 23 20 20

Block 4:

Datenbytes 37 17 56 17 26 52 06 F6 E6 52 E2 20 EC 11 EC 11
Korrekturbytes 8F 7A 07 9D 91 38 00 C6 D8 F1 D3 E1 98 5A BD 7A 86 0B

Vermischen der Daten

Zuletzt werden die Blöcke noch untereinander vermischt. Dann wird folgendermaßen vorgegangen:

| 1. Byte vom 1. Block |
| 1. Byte vom 2. Block |

...

| 1. Byte vom letzten Block |
| 2. Byte vom 1. Block |

...

| letztes Byte vom 1. Block |
| letztes Byte vom 2. Block |

...

| letztes Byte vom letzten Block |
| 1. Korrekturbyte vom 1. Block |
| 1. Korrekturbyte vom 2. Block |

...

| 2. Korrekturbyte vom 1. Block |

...

| letztes Korrekturbyte vom letzten Block |

Achten Sie darauf, dass Blöcke der zweiten Gruppe ein Byte mehr enthalten, die Blöcke der ersten Gruppe also früher aufgeteilt sind!

Beispiel

43 22 36 37 82 02 F6 17 24 D2 46 56 86 02 92 17 56 24 E2 26 C6 76 05 52 C6 56 96 06 F2 E6 F7 F6 07 57 52 E6 46 26 06 52 86 16 17 E2 57 C2 26 20 26 05 52 EC

52 15 06 11 E2 24 12 EC 07 11 (Ende Datenbytes) 60 31 A3 8F B9 21 FC 7A 88 4F 44 07 5F 43 AD 9D AD B3 2C 91 14 46 39 38 E9 AD B1 00 C9 68 7E C6 61 7D 08 D8

44 09 2F F1 8E AC AA D3 96 7A 77 E1 57 20 6B 98 C6 AE 47 5A 74 F2 2F BD A3 C7 23 7A E3 C8 20 86 CE 7E 20 0B

Auswählen der Maske

Zum Auswählen der Maske wird für den von einer Maske erzeugten QR-Code eine Strafpunktzahl nach den folgenden Kriterien berechnet. Mit erzeugtem QR-Code ist wirklich der fertige Code gemeint, also mit Daten und Format-Information und allem anderen Patterns. Die Maske mit der geringsten Summe der Strafpunkte wird dann für den fertigen QR-Code genutzt. Die Beispielbilder sind für den Version 1-L Code "Hallo Welt!" mit der Maske 010.

1. Gruppen von fünf gleichfarbigen Modulen in einer Reihe bzw. Spalte erhalten 3 Strafpunkte und +1 Strafpunkt für jedes weitere gleichfarbige Modul in dieser Gruppe.

2. 2x2 Blöcke mit derselben Farbe bekommen 3 Strafpunkte. Achten Sie darauf, dass wirklich jeder 2x2 Block zählt, also ein 3x3 Block sind 4 2x2 Blöcke!

3. Das Muster schwarz, weiß, schwarz, schwarz, schwarz, weiß, schwarz (die mittlere Zeile des Orientation Patterns) gibt 40 Strafpunkte, wenn davor mindestens 4 weiße Module oder danach mindestens 4 weiße Module sind. Wenn beides der Fall ist, werden die Strafpunkte nur einmal vergeben. Diese Regel wird sowohl vertikal als auch horizontal angewendet.

4. Der folgende Wert als int: 10 * (abs(2 * #DunkleModule - #Module) * 10 / #Module), also eine besonders gewichtete Abweichung zu 50% schwarzer Module.

Achtung: Die Implementation dieser Kriterien ist nicht immer exakt gleich (der Standard ist an dieser Stelle nicht eindeutig), sollten Sie also mit Generatoren aus dem Internet Codes generieren, können diese eventuell anders aussehen, weil sie eine andere Maske nutzen. Gelesen werden können diese von unserer Implementation natürlich trotzdem.
Bei den Testdateien steht immer der zu erhaltene Score dabei, damit können Sie ihre Implementation vergleichen.

Gegebene Klassen
Es sind zur Vereinfachung einige Klassen gegeben. Diese Klassen enthalten hauptsächlich Konstanten aus der QR-Spezifikation bzw. definieren Methoden, um Ihnen Arbeit abzunehmen.

Diese Klassen sind in der gegebenen .jar in lib/ enthalten.

Die Enum ErrorCorrection

Diese Enum stellt die möglichen Fehlerkorrekturlevel der Daten im QR-Code dar. Sie enthält die Konstanten LOW, MEDIUM, QUARTILE und HIGH.

Das Functional Interface MaskFunction

Definiert die Interface-Methode boolean mask(int i, int j) für die Klasse MaskPattern.

Die Enum MaskPattern

Die Konstanten dieser Enum entsprechen den verschiedenen Mustern, die zur Maskierung des QR-Codes verwendet werden können. Jedes Muster besitzt eine Bedingung zur Maskierung.

Feldbezeichner Bedingung

MASK000 (i + j) % 2

MASK001 i % 2

MASK010 j % 3

MASK011 (i + j) % 3

MASK100 (i / 2 + j / 3) % 2

MASK101 (i * j) % 2 + (i * j) % 3

MASK110 ((i * j) % 3 + i * j) % 2

MASK111 ((i * j) % 3 + (i + j) % 2) % 2

Außerdem hat sie die folgende Methode:

public MaskFunction maskFunction()
Gibt eine MaskFunction zurück, die genau dann true zurück gibt, wenn die für diese Instanz oben definierte Bedingung 0 ist.

Die Klasse ErrorCorrectionGroup

Diese Klasse liefert Informationen zu einer Korrekturgruppe (siehe Formatierung der Daten). Sie müssen von dieser Klasse keine Instanzen erstellen. Sie hat die folgenden Methoden:

public int blockCount()
Liefert die Anzahl Blöcke in dieser Gruppe.
public int dataByteCount()
Liefert die Anzahl Datenbytes jedes Blocks dieser Gruppe.

Die Klasse ErrorCorrectionInformation

Diese Klasse liefert Informationen zur Aufteilung in Blöcke der Daten (siehe Formatierung der Daten). Sie müssen von dieser Klasse keine Instanzen erstellen. Sie hat die folgenden Methoden:

public int correctionBytesPerBlock()
Liefert die Anzahl Korrekturbytes jedes Blocks.
public ErrorCorrectionGroup[] correctionGroups()
Liefert die Korrekturgruppen (min. 1, max. 2). Und die folgenden Hilfsmethoden, zum Erstellen von Arrays, deren Längen vorher bekannt sind. Nutzen Sie diese Methoden, Sie brauchen keine dynamisch wachsende Datenstrukturen.
public int totalBlockCount()
Liefert die Anzahl Blöcke (nicht Gruppen!).
public int totalByteCount()
Liefert die Byteanzahl, die alle Daten- und Korrekturbytes zusammen brauchen.
public int lowerDataByteCount()
Liefert die Datenbyteanzahl der Gruppe mit den wenigeren Datenbytes (also der 1. Gruppe).
public int totalDataByteCount()
Liefert die Datenbyteanzahl, die alle Datenbytes zusammen brauchen.

Die Klasse Version

Diese Klasse liefert alle zur Version gehörigen Informationen. Sie definiert die folgenden Methoden, die Sie durch die ganze Aufgabe brauchen werden:

public int number()
Liefert die Versionsnummer dieser Version (1-40).
public int size()
Liefert die Größe eines QR-Codes dieser Version. Z.B. 21 für Version 1.
public int[] alignmentPositions()
Liefert die Positionen für die Alignment Patterns wie oben, also für Version 8 {6, 24, 42}.
public ErrorCorrectionInformation correctionInformationFor(ErrorCorrection level)
Liefert für das Fehlerkorrekturlevel level die zugehörige ErrorCorrectionInformation für diese Version.
public int totalByteCount()
Liefert die Anzahl Bytes (Daten + Korrektur), die ein QR-Code dieser Version enthält.

Und die folgenden statischen Methoden:

public static Version fromNumber(int number)
Liefert die Version -Instanz für diese Versionsnummer (1-40). Nutzen Sie diese Methode, um Instanzen dieser Klasse zu bekommen. Der Konstruktor soll nicht aufgerufen werden (ist sowieso privat).
public static Version forDataBytesCount(int count, ErrorCorrection level)
Liefert die kleinste Version -Instanz, die die gewünschte Anzahl an Datenbytes mit diesem Fehlerkorrekturlevel enthalten kann.

Die Klasse VersionInformation

Diese Klasse hilft Ihnen mit den Version-Information Blöcken. Die Information ist in einem int in den 18 niedrigwertigsten Bits enthalten. Wenn Sie Methoden dieser Klasse nutzen, müssen Sie sich auch an dieses Format halten. Die Klasse hat die folgenden Methoden:

public static int forVersion(Version version)
Liefert die Version-Information für diese Version im oben spezifizierten Format.
public static Version fromBits(int value)
Bestimmt die Version aus den übergebenen Bits im oben spezifizierten Format.

Die Klasse FormatInformation

Diese Klasse hilft Ihnen mit den Format-Information Blöcken. Die Information ist in einem int in den 15 niedrigwertigsten Bits enthalten. Wenn Sie Methoden dieser Klasse nutzen, müssen Sie sich auch an dieses Format halten. Die Klasse hat die folgenden Methoden:

public ErrorCorrection errorCorrection()
Liefert das Fehlerkorrekturlevel dieser FormatInformation.
public MaskPattern maskPattern()
Liefert die Maske dieser FormatInformation.
public int formatInfo()
Liefert die Format-Information dieser FormatInformation.

Und die folgenden statischen Methoden:

public static FormatInformation get(ErrorCorrection errorCorrection, MaskPattern pattern) Liefert die FormatInformation -Instanz für dieses Fehlerkorrekturlevel und diese Maske.
public static FormatInformation fromBits(int value)
Liefert die FormatInformation -Instanz für die gegebenen Format-Information Bits.

Die Klasse reedsolomon.ReedSolomonException

Diese Exception-Klasse wird von der nächsten Klasse geworfen.

Die Klasse reedsolomon.ReedSolomon

Diese Klasse führt die Reed-Solomon Codierung bzw. Dekodierung durch. Sie hat die folgenden Methoden:

public static void correct(byte[] dataBytes, byte[] correctionBytes) throws ReedSolomonException
Korrigiert die übergebenen Daten und Korrekturbytes in-place. Wirft eine ReedSolomonException, sollte irgendein Fehler beim Dekodieren auftreten.

public static byte[] calculateCorrectionBytes(byte[] dataBytes, int correctionBytes)
Berechnet die gewünschte Anzahl Korrekturbytes für die übergebenen Datenbytes.

Aufgabe 1: Vorspiel
Als Erstes werden einige Hilfsklassen im Paket jpp.qrcode definiert und ein Validator für die Codes geschrieben.

Die Enum Encoding

Diese Enum stellt die möglichen Codierungen der Daten im QR-Code dar. Die Enum soll die Felder NUMERIC (0b1), ALPHANUMERIC (0b10), BYTE (0b100), KANJI (0b1000), ECI (0b111), INVALID (-1) beinhalten. Außerdem soll sie die folgenden Methoden haben.

public static Encoding fromBits(int i)
Gibt das nach den gegebenen Binärwerten spezifizierte Encoding zurück, das zur Eingabe passt und sonst INVALID.

public int bits()
Gibt die oben für dieses Encoding definierte Zahl zurück.

Die Klasse QRCodeException

Diese Klasse bildet die Basisklasse für alle Exceptions in dieser Aufgabe. Sie erbt von RuntimeException und implementiert alle Konstruktoren, die Sie benötigen.

Die Klasse InvalidQRCodeException

Diese Klasse erbt von QRCodeException und implementiert alle Konstruktoren, die Sie benötigen. Sie wird geworfen, wenn beim Validieren des QR-Codes ein Fehler auftritt.

Die Klasse QRCode

Diese Klasse repräsentiert einen QR-Code. Sie hat den folgenden Konstruktor:

public QRCode(boolean[][] matrix, Version version, MaskPattern pattern, ErrorCorrection correction)

Und die folgenden Methoden, die die im Konstruktor übergebenen Daten zurückgeben:

public boolean[][] data()
public Version version()
public MaskPattern maskPattern()
public ErrorCorrection errorCorrection()

Außerdem besitzt sie die folgende Methode:

public String matrixToString()
Erstellt einen String für die Matrix des QR-Codes. Für true soll der Charakter (char)0x2588 und für false der Charakter (char)0x2591 jeweils zweimal hintereinander geschrieben werden. Zeilen sollen mit \n getrennt werden.

public static QRCode createValidatedFromBooleans(boolean[][] data)
Diese Methode überprüft, dass der QR-Code richtig formatiert ist und gibt dann eine neue Instanz mit den herausgefundenen Daten aus. Sollte eine Bedingung nicht erfüllt sein,
soll eine InvalidQRCodeException mit einer aussagekräftigen Fehlermeldung geworfen werden.

Die Matrix ist nicht null, nicht leer und quadratisch.
Die Größe der Matrix passt zu einer validen QR-Code Version.
Alle oben unter "Layout" genannten Bedingungen für einen validen QR-Code sind erfüllt.
Für Codes mit Version > 6: (Mindestens) einer der beiden Version-Information Blöcke ist valid, d.h. VersionInformation.fromBits hat einen validen Wert (nicht null) zurückgegeben und gibt die gleiche Versionsnummer, wie die vorher durch die Größe der Matrix bestimmte an.
(Mindestens) einer der beiden Format-Information Blöcke ist valid, d.h. die Methode FormatInformation.fromBits hat einen validen Wert (nicht null) zurückgegeben. Ist
dies der Fall, so enthält diese FormatInformation Instanz auch die gesuchte MaskPattern und ErrorCorrection des QR-Codes.

Die Klasse ReservedModulesMask

Diese Klasse soll für eine gegebene Position true zurückgeben, wenn diese Position durch ein Pattern oder den Version-/Formatinformation Block (also alles außer den Daten) reserviert ist. Sie soll das spätere Auslesen und Einfügen der Daten erleichtern.
Sie soll den folgenden Konstruktor haben:

public ReservedModulesMask(boolean[][] mask)
Erstellt eine ReservedModulesMask, wobei genau dann an einer Position ein reserviertes Feld ist, wenn die übergebene Maske an dieser Position den Wert true enthält. Diese Maske soll durch diese Klasse nicht verändert werden können. Sie können davon ausgehen, dass mask nicht null ist und sollen keine Kopie von mask machen. Sie können davon ausgehen, dass die Matrix quadratisch ist.

public boolean isReserved(int i, int j)
Gibt zurück, ob das Modul an der Position (i,j) reserviert ist.

public int size()
Gibt zurück, wie groß (in einer Dimension) diese Maske ist. Die gesamte Maske ist also size() x size() groß.

public static ReservedModulesMask forVersion(Version version)
Erstellt eine ReservedModulesMask für die übergebene Version. Nutzen Sie dazu die Methoden Version.size() und Version.alignmentPositions(). Achten Sie darauf, dass
Ihre erstellte Matrix quadratisch ist.

Die Klasse MaskApplier

Diese Klasse soll eine Maske auf eine Matrix anwenden. Sie sollte in den folgenden Aufgaben genutzt werden und implementiert die folgende Methode:

public static void applyTo(boolean[][] data, MaskFunction mask, ReservedModulesMask reservedModulesMask)
Invertiert jede Position von data, an der die gegebene MaskFunction true zurück gibt und reservedModulesMask bestätigt, dass dieses Modul nicht reserviert ist. Sie können davon ausgehen, dass keiner der Parameter null ist und data quadratisch ist. Sollte die Größe von data nicht zur Größe von der ReservedModulesMask passen, soll eine IllegalArgumentException geworfen werden.

Die Klasse DataBlock

Diese Klasse enthält die Daten- und Korrekturbytes eines Blocks (siehe Formatierung der Daten). Sie hat den folgenden Konstruktor:

public DataBlock(byte[] dataBytes, byte[] correctionBytes) Erstellt einen DataBlock mit die übergebenen Arrays. Sie sollen keine Kopien zu erstellen.

Die folgenden Methoden, geben die im Konstruktor übergebenen Daten zurück:

public byte[] dataBytes()
public byte[] correctionBytes()

Die Klasse DataPositions

Diese Klasse soll die Reihenfolge, in der die Daten ausgelesen werden (das Zickzackmuster) unter Berücksichtigung einer ReservedModulesMask bestimmen. Sie verhält sich im Prinzip wie ein Iterator.
Die Klasse hat den folgenden Konstruktor:

public DataPositions(ReservedModulesMask mask)
Nutzt die übergebene Maske und setzt die Position auf das erste Feld. Sie können davon ausgehen, dass mask nicht null ist und das erste Feld immer unten rechts ist, nicht reserviert ist und existiert (d.h. mask.size() > 0).

Sie hat außerdem die folgenden Methoden:

public int i()
Gibt die Koordinate in i -Richtung zurück.

public int j()
Gibt die Koordinate in j -Richtung zurück.

public boolean next()
Versucht, die nächste Datenposition zu finden und setzt i und j dementsprechend. Sollte keine gefunden werden, wird false zurückgegeben, sonst true. Vergessen Sie nicht,
die Spalte mit dem Index 6 zu überspringen.
Diese Funktion darf nur aufgerufen werden, wenn ein vorheriger Aufruf von next true zurückgegeben hat oder noch keiner erfolgt ist. Sonst ist ein Aufruf dieser Funktion nicht erlaubt, Sie können diesen Fall also ignorieren.

Beispiel zur Benutzung

DataPositions dataPositions = new DataPositions(mask);

do {
int i = dataPositions.i();
int j = dataPositions.j();
} while(dataPositions.next());

Aufgabe 2: Lesen und schreiben
Diese Aufgabe beschäftigt sich mit einem kleinen Reader bzw. Writer für Textdateien, damit man leicht QR-Codes in das Programm laden kann.
Alle Dateien sind im Paket jpp.qrcode.io abzulegen.

Die Klasse TextReader

Diese Klasse dient zum Einlesen von QR-Codes in Textform, wie auch die gegebenen Textdateien und soll die folgende statische Methode enthalten:

public static boolean[][] read(InputStream in) throws IOException
Liest die Matrix aus dem InputStream aus und gibt einen damit erzeugten QR-Code zurück.
Die Dateien liegen als Matrizen aus 1en und 0en in Textform vor und enthalten Kommentarzeilen, die mit # beginnen und ignoriert werden sollen. Leerzeichen und leere Zeilen sollen ebenfalls ignoriert werden. Falls der Eingabestrom keine Daten oder andere Zeichen als 1, 0, Whitespace und Kommentarzeilen enthält, so soll eine aussagekräftige IOException geworfen werden.

Die Klasse TextWriter

Diese Klasse dient zum Schreiben von QR-Codes in Textform, wie auch die gegebenen Textdateien und soll die folgende statische Methode enthalten:

public static void write(OutputStream stream, boolean[][] data) throws IOException
Schreibt die Matrix in dem in der vorherigen Klasse beschriebenen Format in out. Für Zeilenumbrüche darf nur '\n' genutzt werden. Sie dürfen beliebige Kommentarzeilen und Leerzeichen/Leerzeilen wo Sie möchten einbauen, es wird nur getestet, dass das Format an sich valide ist, nicht dass exakt dasselbe wie von der Musterlösung ausgegeben
wird.

Hinweis: Alle Klassen in den Aufgaben 3 und 4 (bis auf die letzte) sind komplett unabhängig voneinander, können also in beliebiger Reihenfolge implementiert werden. Außerdem invertieren sich jeweils die Klassen DataEncoder & DataDecoder, DataStructurer & DataDestructurer und DataExtractor & ModulePlacer, können also super lokal mit Dummy-Eingaben getestet werden.
Achten Sie darauf, eine sehr effiziente Implementation zu erstellen, die Timeouts der Tests für die folgenden Klassen sind mit Absicht knapp gewählt.

Aufgabe 3: Dekodierung
Ziel dieser Aufgabe ist es, einen QR-Code zu dekodieren. Dazu wird zuerst der Code validiert und eine ReservedModulesMask für dessen Version erstellt. Dann werden die Datenbits mit der entsprechenden Maske demaskiert und mit der Klasse DataExtractor aus dem Code extrahiert. Die Klasse DataDestructurer macht die Vermischung der Daten rückgängig und führt die Fehlerkorrektur aus. Abschließend decodiert der DataDecoder die Daten.
Alle Dateien sind im Paket jpp.qrcode.decode abzulegen.

Die Klasse QRDecodeException

Diese Klasse erbt von QRCodeException und implementiert alle Konstruktoren, die Sie benötigen. Sie wird geworfen, wenn beim Dekodieren ein Fehler auftritt.

Die Klasse DataExtractor

Diese Klasse soll die Daten aus einem demaskierten QR-Code extrahieren. Sie implementiert die folgende Methode:

public static byte[] extract(boolean[][] data, ReservedModulesMask mask, int byteCount)
Liest die Bits in der oben genannten Reihenfolge aus data aus und fügt sie in der ausgelesenen Reihenfolge in einen byte[] ein. Die Größe des byte[] ist gegeben durch byteCount, Sie benötigen also keine dynamisch wachsende Datenstruktur.
Nutzen Sie zum Auslesen DataPositions. Außerdem haben manche Versionen 7 Bits mehr als die spezifizierte Byteanzahl, diese können aber ignoriert werden (sind sowieso
0). Brechen Sie also das Auslesen ab, sobald die gesuchte Byteanzahl ausgelesen wurde. Sie können davon ausgehen, dass data und mask nicht null sind, byteCount
nicht negativ und data quadratisch ist.
Sollte die ReservedModulesMask nicht zur Größe von data passen, soll eine IllegalArgumentException geworfen werden.

Die Klasse DataDestructurer

Diese Klasse soll die extrahierten Daten nach dem oben beschriebenen Muster umstrukturieren. Es muss also zuerst das Vermischen und dann das Aufteilen in Blöcke rückgängig gemacht werden.
Alle Funktionen dieser Klasse nehmen ErrorCorrectionInformation entgegen, lesen Sie sich also die Informationen dazu oben durch. Sie können davon ausgehen, dass alle Argumente nicht null sind und die ErrorCorrectionInformation zu dem DataBlock[] bzw. dem byte[] passen. Implementieren Sie die folgenden Methoden.

public static byte[] join(DataBlock[] blocks, ErrorCorrectionInformation errorCorrectionInformation)
Überprüft bzw. korrigiert mit Hilfe der Korrekturbytes die Blöcke und fügt sie wieder zu den eigentlichen Datenbytes zusammen. Zur Korrektur wird ReedSolomon.correct aufgerufen.
Sollte eine ReedSolomonException geworfen werden, soll eine QRDecodeException mit aussagekräftiger Nachricht geworfen werden.
Die korrigierten Datenbytes (nicht die Korrekturbytes) sollen in der Reihenfolge der Blöcke zu einem byte[] wieder zusammengefügt werden, dessen Größe durch ErrorCorrectionInformation.totalDataByteCount bereits bekannt ist.

public static DataBlock[] deinterleave(byte[] data, ErrorCorrectionInformation errorCorrectionInformation)
Macht das Vermischen der Daten rückgängig, d.h. sortiert die Daten wieder nach den gegebenen errorCorrectionInformation in Blöcke und gibt diese zurück. Die Anzahl Blöcke ist durch ErrorCorrectionInformation.totalBlockCount gegeben.

public static byte[] destructure(byte[] data, ErrorCorrectionInformation ecBlocks)
Diese Funktion soll der Einstiegspunkt für diese Klasse sein. Sie soll überprüfen, dass der byte[] so groß ist, wie ecBlocks dies erwartet und sonst eine IllegalArgumentException werfen. Dann ruft sie die beiden anderen Funktionen auf.

Die Klasse DataDecoder

Diese Klasse decodiert die korrigierten Daten des QR-Codes. Sie implementiert die folgenden Methoden:

public static Encoding readEncoding(byte[] bytes)
Liest die ersten 4 Bits aus dem Array und gibt das Encoding zurück, das Encoding.fromBits zurückgibt. Sie können davon ausgehen, dass der Array mindestens so viele Elemente wie benötigt enthält.

public static int readCharacterCount(byte[] bytes, int count)
Liest die gegebene Bitanzahl (8 oder 16) aus dem Array ab dem Bit 4 und gibt die resultierende Zahl zurück. Sie können davon ausgehen, dass der Array mindestens so viele Elemente wie benötigt enthält.

public static String decodeToString(byte[] bytes, Version version, ErrorCorrection errorCorrection)
Dekodiert die übergebenen Bytes in einen String. Brechen sie mit einer IllegalArgumentException ab, wenn die Byteanzahl nicht zur Datenbyteanzahl dieser Version mit dem gegebenen Fehlerkorrekturlevel passt. Werfen Sie außerdem eine QRDecodeException, wenn das Encoding nicht BYTE ist oder der Character count mehr Bytes angibt, als
gelesen werden können.
Sie brauchen die Version des QR-Codes, von dem die Daten stammen, da die Zeichenanzahl unterschiedliche Bitlängen hat.
Nutzen Sie die vorherigen beiden Funktionen, um Fehler zu vermeiden.
Dekodieren Sie nun so viele Bytes, wie der Character Count Indicator angibt und erstellen Sie einen String mit dem Encoding ISO-8859-1 daraus.

Die Klasse Decoder

Diese Klasse fügt alle vorher definierten Klassen zusammen. Sie implementiert die folgende Methode:

public static String decodeToString(QRCode qrCode)
Erstellt eine Kopie von der Matrix des QR-Codes, demaskiert diese (also wendet die Maske darauf an), extrahiert die Daten und dekodiert diese (DataDestructurer und DataDecoder).

Aufgabe 4: Kodierung
Ziel dieser Aufgabe ist es, einen QR-Code zu erstellen. Dazu wird die Nachricht zuerst kodiert, in Blöcke aufgeteilt und die Blöcke vermischt. Anschließend werden die Daten in den vorbereiteten QR-Code platziert, alle Masken durchprobiert und evaluiert und die beste ausgesucht.
Alle Dateien sind im Paket jpp.qrcode.encode abzulegen.

Die Klasse DataEncoderResult

Da der Kodierungsschritt sowohl die Bytes als auch die Version des Codes bestimmt, brauchen wir eine Hilfsklasse für den Rückgabewert. Diese Klasse hält genau diese beiden Informationen. Es sollen keine Kopien gemacht werden. Sie hat den folgenden Konstruktor:

public DataEncoderResult(byte[] bytes, Version version)

Und die folgenden Methoden, die die im Konstruktor übergebenen Daten zurückgeben:

public byte[] bytes()
public Version version()

Die Klasse DataEncoder

Diese Klasse kodiert die Eingabedaten nach dem oben beschriebenen Byteformat.
Sie implementiert die folgende Methode:

public static DataEncoderResult encodeForCorrectionLevel(String str, ErrorCorrection level)
Kodiert die Daten nach dem oben beschriebenen Byteformat. Nutzen Sie die Funktion Version.forDataBytesCount eventuell mehrmals. Vergessen Sie nicht, dass die Länge
des Character Count Indicators unterschiedlich sein kann, die Daten terminiert und mit 0 bzw. Padding Bytes aufgefüllt werden müssen.

Die Klasse DataStructurer

Diese Klasse führt das oben beschriebene Aufteilen in Blöcke und das Vermischen der Daten durch.
Alle Funktionen dieser Klasse nehmen ErrorCorrectionInformation entgegen, lesen Sie sich also die Informationen dazu oben durch. Sie können davon ausgehen, dass alle Argumente nicht null sind und die ErrorCorrectionInformation zu den DataBlocks bzw. dem byte[] passen.
Implementieren Sie die folgenden Methode:

public static DataBlock[] split(byte[] data, ErrorCorrectionInformation errorCorrectionInformation)
Teilt die Daten nach den gegebenen errorCorrectionInformation in Datenblöcke, deren Anzahl durch ErrorCorrectionInformation.totalBlockCount vorgegeben ist, wie oben erklärt, auf und bestimmt deren Korrekturbytes mit ReedSolomon.calculateCorrectionBytes. Achten Sie darauf, dass die zweite Gruppe mehr Datenbytes enthält.

public static byte[] interleave(DataBlock[] blocks, ErrorCorrectionInformation ecBlocks) Vermischt die Blöcke wie oben beschrieben. Die Größe des byte[] ist
durch ErrorCorrectionInformation.totalByteCount gegeben.

public static byte[] structure(byte[] data, ErrorCorrectionInformation ecBlocks)
Diese Funktion soll der Einstiegspunkt für diese Klasse sein. Sie soll überprüfen, dass der byte[] so groß ist, wie ecBlocks dies erwartet und sonst eine IllegalArgumentException mit aussagekräftiger Nachricht werfen. Dann ruft sie die beiden anderen Funktionen auf.

Die Klasse PatternPlacer

Diese Klasse platziert die Patterns in den QR-Code, wie oben beschrieben. Die Idee dabei ist es, einen rohen QR-Code zu erstellen, in den dann die Daten unmaskiert geschrieben werden. Dieser rohe QR-Code wird dann zum Auswählen/Durchprobieren der Maske genutzt, es muss also nicht der komplette bisherige Erstellungsvorgang wiederholt werden. Sie können bei jeder der Methoden davon ausgehen, dass die Argumente nicht null sind und die Größe der Matrix zur Version passt.

public static void placeOrientation(boolean[][] res, Version version)
Fügt die Orientation-Patterns für die gegebene Version in res ein.

public static void placeTiming(boolean[][] res, Version version)
Fügt die Timing-Patterns für die gegebene Version in res ein.

public static void placeAlignment(boolean[][] res, Version version)
Fügt die Alignment-Patterns für die gegebene Version in res ein.

public static void placeVersionInformation(boolean[][] data, int versionInformation)
Fügt die gegebene Versionsinformation in res ein. Es gelten die gleichen Regeln zu versionInformation wie in der Klasse VersionInformation beschrieben.

public static boolean[][] createBlankForVersion(Version version)
Erstellt einen leeren QR-Code für die gegebene Version. Das heißt er enthält nur die folgenden Patterns und ist sonst leer:

Alle Orientation-Patterns,
alle Timing-Patterns,
alle Alignment-Patterns,
das Dark Module und
eventuell die Version-Information.

Die Klasse DataInserter

Diese Klasse platziert die Daten unmaskiert im QR-Code. Implementieren Sie die folgenden Methode:

public static void insert(boolean[][] target, ReservedModulesMask mask, byte[] data)
Fügt die Daten unmaskiert in target nach der gegebenen ReservedModulesMask ein. Sie können davon ausgehen, dass die Daten bis auf die möglichen 7 übrigen Bits reichen
und können deshalb, sobald alle Daten geschrieben wurden, aufhören.

Die Klasse MaskSelector

Diese Klasse berechnet die Strafpunktzahlen für QR-Codes und nutzt die Ausgabe der vorherigen Klasse, probiert darauf alle Masken aus und wählt die beste passende.

public static void placeFormatInformation(boolean[][] res, int formatInformation)
Diese Funktion könnte zwar auch in der vorherigen Klasse sein, aber da die Klassen möglichst unabhängig implementiert werden können sollen und sie nur von dieser Klasse genutzt wird, ist sie hier. Fügt die gegebene Format-Information in res ein. Es gelten die gleichen Regeln zu formatInformation wie in der Klasse FormatInformation beschrieben.
Achten Sie darauf, dass Sie alle Werte (sowohl true als auch false) in die dementsprechenden Module schreiben, da sonst maskWithBestMask mit dem beschriebenen Algorithmus nicht funktioniert.

public static int calculatePenaltySameColored(boolean[][] data)
Berechnet die erste Strafpunktzahl wie oben beschrieben.

public static int calculatePenalty2x2(boolean[][] arr)
Berechnet die zweite Strafpunktzahl wie oben beschrieben.

public static int calculatePenaltyBlackWhitePattern(boolean[][] array)
Berechnet die dritte Strafpunktzahl wie oben beschrieben.

public static int calculatePenaltyWeightedDeviation(boolean[][] arr)
Berechnet die vierte Strafpunktzahl wie oben beschrieben.

public static int calculatePenaltyFor(boolean[][] data)
Berechnet die Summe der vier Strafpunktzahlen.

public static MaskPattern maskWithBestMask(boolean[][] data, ErrorCorrection correction, ReservedModulesMask modulesMask)
Wählt die beste Maske aus und maskiert damit die QR-Code Daten.
Dazu soll für alle Masken nach den unten genannten Schritten vorgegangen werden. Danach soll die am besten bewertete Maske wieder auf die Matrix angewendet, die korrekte Format-Information eingefügt und diese Maske zurückgegeben werden.
Sollte die ReservedModulesMask nicht zur Größe der Daten passen, soll eine IllegalArgumentException mit aussagekräftiger Nachricht geworfen werden.
Sollten mehrere Masken die gleiche Strafpunktzahl ergeben, wählen Sie die Maske mit dem kleineren Index (also die, die zuerst in obiger Tabelle vorkommt).

Bestimmen Sie die Format-Information für diese Maske und das Fehlerkorrekturlevel.
Wenden Sie die Maske auf die Matrix an (MaskApplier).
Fügen Sie die Format-Information ein.
Berechnen Sie die Strafpunktzahl.
Entmaskieren Sie die Matrix wieder.

Die Klasse Encoder

Diese Klasse fügt alle vorher definierten Klassen zusammen. Sie implementiert die folgende Methode:

public static QRCode createFromString(String msg, ErrorCorrection correction)
Kodiert die Daten (DataEncoder und DataStructurer), erstellt einen leeren QR-Code für die benötigte Version, fügt die kodierten Daten unmaskiert ein, bestimmt die beste Maske und erstellt aus allen gesammelten Daten einen QRCode.
Achten Sie darauf, die ReservedModulesMask nicht mehrmals zu erstellen, da dies je nach Implementation sehr teuer sein kann.

Hinweis: Einige Tests haben Ausgaben mit einer Zahl und einem Datensatz-Namen (z.B. 123 (Normal)). Diese Informationen können hier eingegeben werden, um den QR-Code in unterschiedlichen Darstellungen zum Reproduzieren der Tests zu erhalten und je nach Datensatz einige Informationen zu den Codes einzusehen (z.B. die Strafpunktzahlen in Penalties).